HCMC University of Technology, VNU

Software Architecture

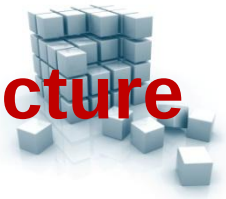


Chapter 6. Documenting Software Architecture

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

Chapter 6. Documenting Software Architecture



6.1. Documenting Software Architecture

6.2. Module Views

6.3. Component-and-Connector Views

6.4. Allocation Views

6.1. Documenting Software Architecture



- **What's difference between architecture and design?**
- ***Architecture is design, but not all design is architectural.***
  - The architect draws the **boundary between architectural and non-architectural design** by making those decisions that **need to be bound** in order for the system to **meet its development, behavioral, and quality goals**.
  - All other decisions can be left to downstream designers and implementers.
- ***Decisions are architectural or not, according to context*.**
  - If structure is **important to achieve** your **system's goals**, that structure is **architectural**.
  - But designers of elements, or subsystems, that you assign may have to introduce **structure of their own to meet their goals**, in which case **such structures are architectural: to them but not to you**.

6.1. Documenting Software Architecture



- **Why Document Software Architecture?**
- Even the best **architecture**, most perfectly suited for the job, will be essentially **useless** if the people **who need to use it do not know what it is**, cannot understand it well enough to apply it, or (worst of all) misunderstand it and apply it incorrectly.
- Creating an architecture isn't enough. It has to be **communicated** in a way **to let its stakeholders use it properly** to do their jobs.
- **Documentation speaks for the architect**. It speaks for the architect today, when the architect should be doing other things besides answering a hundred questions about the architecture. And it speaks for the architect tomorrow, when he or she has left the project and now someone else is in charge of its evolution and maintenance.

6.1. Documenting Software Architecture



- **Why Document Software Architecture?**
- The best architects produce the best documentation **not because it's "required," but because** they see that it is **essential** to the matter at hand: **producing a high-quality product**, **predictably** and **with as little rework as possible**.
- The best architects also **see documentation as delivering value to themselves**. Documentation serves as the receptacle to hold the results of design decisions as they are made. A well-thought-out documentation scheme can make the process of design go much more smoothly and systematically.

6.1. Documenting Software Architecture



- **Uses and Audiences for Architecture Documentation**
- Architecture documentation must **serve varied purposes**. It should be sufficiently **abstract** to **be quickly understood** by new employees. It should be sufficiently **concrete** to serve as a **blueprint for construction**. It should have **enough information** to serve as a **basis for analysis**.
- Architecture documentation is both **prescriptive** and **descriptive**. For some audiences, it **prescribes what should be true**, placing **constraints** on decisions yet to be made. For other audiences, it **describes what is true**, recounting decisions already made about a system's design.

6.1. Documenting Software Architecture



- **Uses and Audiences for Architecture Documentation**
- Fundamentally, architecture documentation has three uses.
  - Architecture serves as a **means of education**.
  - Architecture serves as a **primary vehicle for communication** among stakeholders.
  - Architecture serves as the **basis for** system **analysis** and **construction**.

6.1. Documenting Software Architecture



Table P.1 Some of the stakeholders of architecture documentation, their roles, and how they might use it

Name	Description	Use for Architecture Documentation
Analyst	Responsible for analyzing the architecture to make sure it meets certain critical quality attribute requirements. Analysts are often specialized; for instance, performance analysts, safety analysts, and security analysts may have well-defined positions in a project.	Analyzing satisfaction of quality attribute requirements of the system based on its architecture.
Architect	Responsible for the development of the architecture and its documentation. Focus and responsibility is on the system.	Negotiating and making trade-offs among competing requirements and design approaches. A vessel for recording design decisions. Providing evidence that the architecture satisfies its requirements.
Business manager	Responsible for the functioning of the business/organizational entity that owns the system. Includes managerial/executive responsibility, responsibility for defining business processes, and more.	Understanding the ability of the architecture to meet business goals.

6.1. Documenting Software Architecture



Name	Description	Use for Architecture Documentation
Conformance checker	Responsible for assuring conformance to standards and processes to provide confidence in a product's suitability.	Basis for conformance checking, for assurance that implementations have been faithful to the architectural prescriptions.
Customer	Pays for the system and ensures its delivery. The customer often speaks for or represents the end user, especially in a government acquisition context.	Assuring required functionality and quality will be delivered, gauging progress, estimating cost, and setting expectations for what will be delivered, when, and for how much.
Database administrator	Involved in many aspects of the data stores, including database design, data analysis, data modeling and optimization, installation of database software, and monitoring and administration of database security.	Understanding how data is created, used, and updated by other architectural elements, and what properties the data and database must have for the overall system to meet its quality goals.
Deployer	Responsible for accepting the completed system from the development effort and deploying it, making it operational, and fulfilling its allocated business function.	Understanding the architectural elements that are delivered and to be installed at the customer's or end user's site, and their overall responsibility toward system function.
Designer	Responsible for systems and/or software design downstream of the architecture, applying the architecture to meet specific requirements of the parts for which they are responsible.	Resolving resource contention and establishing performance and other kinds of runtime resource consumption budgets. Understanding how their part will communicate and interact with other parts of the system.

6.1. Documenting Software Architecture



Name	Description	Use for Architecture Documentation
Evaluator	Responsible for conducting a formal evaluation of the architecture (and its documentation) against some clearly defined criteria.	Evaluating the architecture's ability to deliver required behavior and quality attributes.
Implementer	Responsible for the development of specific elements according to designs, requirements, and the architecture.	Understanding inviolable constraints and exploitable freedoms on development activities.
Integrator	Responsible for taking individual components and integrating them, according to the architecture and system designs.	Producing integration plans and procedures, and locating the source of integration failures.
Maintainer	Responsible for fixing bugs and providing enhancements to the system throughout its life (including adaptation of the system for uses not originally envisioned).	Understanding the ramifications of a change.
Network administrator	Responsible for the maintenance and oversight of computer hardware and software in a computer network. This may include the deployment, configuration, maintenance, and monitoring of network components.	Determining network loads during various use profiles and understanding uses of the network.

6.1. Documenting Software Architecture



Name	Description	Use for Architecture Documentation
Product line manager	Responsible for development of an entire family of products, all built using the same core assets (including the architecture).	Determining whether a potential new member of a product family is in or out of scope and, if out, by how much.
Project manager	Responsible for planning, sequencing, scheduling, and allocating resources to develop software components and deliver components to integration and test activities.	Helping to set budget and schedule, gauging progress against established budget and schedule, and identifying and resolving development-time resource contention.
Representative of external systems	Responsible for managing a system with which this one must interoperate, and its interface with our system.	Defining the set of agreement between the systems.
System engineer	Responsible for design and development of systems or system components in which software plays a role.	Assuring that the system environment provided for the software is sufficient.
Tester	Responsible for the (independent) test and verification of the system or its elements against the formal requirements and the architecture.	Creating tests based on the behavior and interaction of the software elements.
User	The actual end users of the system. There may be distinct kinds of users, such as administrators, superusers, and so on.	Users, in the role of reviewers, might rely on architecture documentation to check whether desired functionality is being delivered. Users might also refer to the documentation to understand what the major system elements are, which can aid them in emergency field maintenance.

6.1. Documenting Software Architecture

The Views and Beyond "Method"



- An architecture document should be **helpful** to the people **who depend on it to do their work**.
- The techniques can be bundled into a few categories:
  - **Finding out** what **stakeholders need**. If you don't do this, you're going to end up with documentation that may serve no one.
  - **Providing** the **information** to **satisfy those needs** by **recording design decisions** according to a variety of **views**, plus the **beyond-view information**.
  - **Checking** the resulting **documentation** to see **if** it **satisfied** the **needs**.
  - **Packaging** the **information** in a **useful form** to its stakeholders.

6.1. Documenting Software Architecture

Architecture Views

- Documenting an architecture is a matter of **documenting** the **relevant views** and then adding **documentation that applies to more than one view**.
- A **view** is a **representation** of a set of system elements and the relationships associated with them.

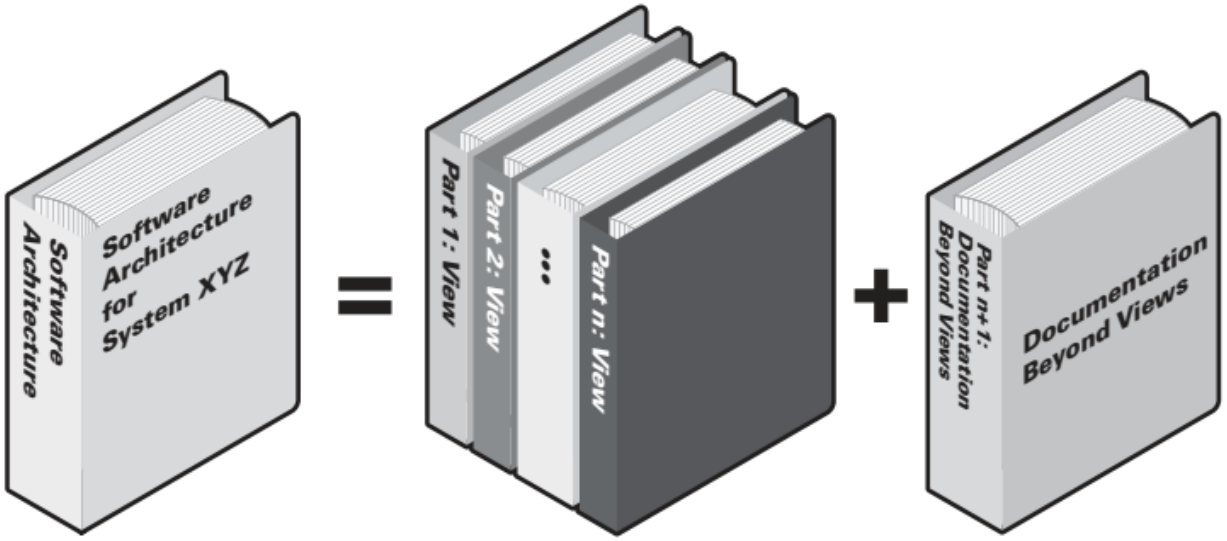


Figure P.1
A documentation package for a software architecture can be composed of one or more view documents and documentation that explains how the views relate to one another, introduces the package to its readers, and guides them through it.

6.1. Documenting Software Architecture



Architecture Views

- **Different views support different goals and uses**. This is fundamentally why we do not advocate a particular view or collection of views.
- The **views** you **should document depend on** the **uses** you **expect to make of the documentation**. **Different views** will **highlight different system elements** and/or **relations**.

6.1. Documenting Software Architecture



Architecture Views

- *The documentation for a view contains*:
  - A primary presentation, usually **graphical**, that depicts the **primary elements** and **relations** of the view.
  - An element catalog that explains and defines the elements shown in the view and lists their properties.
  - A specification of the **elements' interfaces** and **behavior**.
  - A variability guide explaining any built-in mechanisms available for tailoring the architecture.
  - **Rationale** and **design information**.

6.1. Documenting Software Architecture



Architecture Views

- The documentation that applies to all of the views contains:
  - An introduction to the **entire package**, including a reader's guide that **helps** a **stakeholder find** a **desired piece of information quickly**.
  - Information describing **how** the **views relate to one another**, and to the system as a whole.
  - **Constraints** and **rationale** for the overall architecture.
  - Such management information as may be required to effectively maintain the whole package.

6.1. Documenting Software Architecture



Architecture Views

- Architects need to think about their software in three ways simultaneously:
  - How it is **structured** as a set of **implementation units**.
  - How it is **structured** as a set of **elements** that have **runtime behavior** and **interactions**.
  - How it relates to **nonsoftware structures** in its **environment**.
- An architecture is commonly documented in at least three views:
  - **Module views**
  - **Component-and-connector (C&C) views**
  - **Allocation views**

6.2. Module Views



- Module views document a **system's principal units of implementation**.
- It also **includes global data structures** that impact and are impacted by multiple units.
- Module structures often determine how changes to one part of a system might affect other parts and hence the ability of a system to support modifiability, portability, and reuse.
- *It is unlikely that the documentation of any software architecture can be complete without at least one module view*.

6.2. Module Views



- Module views **enumerate** the **principal implementation units**, or **modules**, of a system, together with the **relations** among these units.
- Module views can be used for each of the **purposes**: **education**, **communication** among stakeholders, and the **basis for construction** and **analysis**.
- The way in which a system's software is decomposed into manageable units remains one of the important forms of system structure.
- At a minimum, it determines **how a system's source code is decomposed into units**, what kinds of assumptions each unit can make about services provided by other units, and how those units are aggregated into larger ensembles.

6.2. Module Views



Table 1.1 Summary of the module views

Elements	Modules, which are implementation units of software that provide a coherent set of responsibilities.
Relations	• *Is part of*, which defines a part/whole relationship between the submodule—the part—and the aggregate module—the whole. • *Depends on*, which defines a dependency relationship between two modules. Specific module styles elaborate what dependency is meant. • *Is a*, which defines a generalization/specialization relationship between a more specific module—the child—and a more general module—the parent.
Constraints	Different module views may impose specific topological constraints.
What It's For	• Providing a blueprint for construction of the code • Facilitating impact analysis • Planning incremental development • Supporting requirements traceability analysis • Explaining the functionality of the system and the structure of the code base • Supporting the definition of work assignments, implementation schedules, and budget information • Showing the structure of information to be persisted

6.2. Module Views



Elements

- A **module** is an **implementation unit** of software that **provides** a **coherent set of responsibilities**.
- A **responsibility** is a **general statement** about an **architecture element** and **what** it is **expected to contribute** to the **architecture**. This includes the **actions** that it **performs**, the **knowledge** it **maintains**, the **decisions** it **makes**, or the **role** it **plays** in **achieving** the system's **overall quality attributes** or functionality.
- Modules can be aggregated and decomposed.

6.2. Module Views



Relations

- Module views have the following **types of relations**:
  - **Is-part-of**: defines a **part/whole relationship** between the **submodule - the part**, and the **aggregate module - the whole**. In its most general form, *the is-part-of relation simply indicates aggregation*, with little implied semantics.
  - **Depends-on**: A depends on B defines a **dependency relation** between A and B. Many different specific forms of dependency can be used in module views. The **depends-on relation is refined to: uses, allowed to use, crosscuts, and data entity relationships**; in the module uses, layered, aspect, and data model styles, respectively. The logical association between classes (in a UML class diagram, for example) also depicts a dependency between the classes.

6.2. Module Views



Relations

- Module views have the following **types of relations**:
  - **Is-a**: defines a **generalization/specialization relationship** between a **more specific module - the child**, and a **more general module - the parent**. The child is able to be used in contexts in which the parent is used. Object-oriented inheritance and interface realization are special cases of the is-a relation.

6.2. Module Views



Properties

- **Properties of modules** that help to **guide implementation** or are **input to analysis** should be recorded as part of the supporting documentation for a module view.
- The **list of properties** may vary but is likely to include the following:
  - **Name**: the primary means to refer to it.
  - **Responsibility**: a way to identify its role in the overall system and establishes an identity for it beyond the name.

6.2. Module Views



Properties

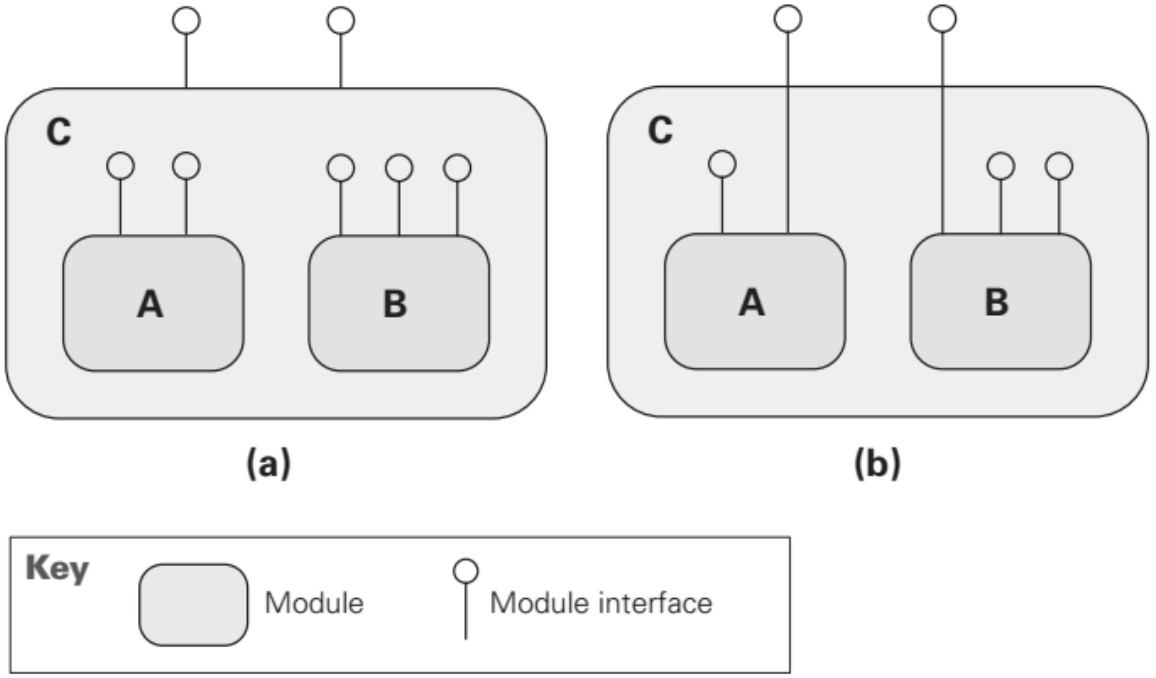
- The **list of properties** may vary but is likely to include the following:
  - **Visibility of interface**(s):
    - **Encapsulation** (Figure 1.1(a)). The parent module provides its own interfaces and maps all requests to the capabilities provided by the submodules. The facilities of the enclosed modules are not available outside the parent.
    - The **interfaces of an aggregate module can be a subset of the interfaces of its submodules** (Figure 1.1(b)). The aggregate module selectively exposes some of the interfaces of the submodules.

6.2. Module Views

Properties

- The **list of properties** may vary but is likely to include the following:
  - **Visibility of interface**(s):

Figure 1.1
(a) Module C provides its own interface, hiding the interfaces of modules A and B. (b) Module C exposes a subset of the interfaces of modules A and B as its interface.



6.2. Module Views



Properties

- The **list of properties** may vary but is likely to include the following:
  - **Implementation information**:
    - **Mapping to source code units**: This identifies the files that constitute the implementation of a module.
    - **Test information**: The module's test plan, test cases, test scaffolding, and test data are important to store.
    - **Management information**: A manager may need information about the module's predicted schedule and budget.
    - **Implementation constraints**: constraints that the implementation must follow.

6.2. Module Views



What Module Views Are For

- **Expect to use module views for**
  - **Construction**: A module view can provide a blueprint for the source code and the data store.
  - **Analysis**: Module views that show dependencies among modules or layers provide a good basis for impact analysis.
  - **Communication**. A module view can be used to explain the system's functionality to someone not familiar with the system.
- On the other hand, it is **difficult** to use the **module views** to make inferences about **runtime behavior**, **because** these views are just a **static partition of the functions** of the software. Thus, a module view is not typically used for analysis of performance, reliability, or many other runtime qualities. For those, we typically rely on component-and-connector and allocation views.

6.2. Module Views



Notations for Module Views

- **Informal Notations**
  - A number of notations can be used to present a module view.
  - One common informal notation uses **boxes** to represent the **modules**, with different kinds of **lines** between them representing the **relations** (box-and-line diagram). **Nesting** is used to depict **aggregation**, and **arrows** typically represent a **depends-on relation**.
  - A second common form of informal notation is a simple **textual listing** of the **modules with descriptions** of the **responsibilities**. Various textual schemes can be used to represent the **is-part-of relation**, such as indentation, outline numbering, and parenthetical nesting.

6.2. Module Views



Notations for Module Views

- **Unified Modeling Language (UML)**
  - Software modeling notations, such as UML, provide a variety of constructs that can be used to represent modules.
  - **UML** has a **class** construct, which is the object-oriented specialization of a module as described here.
  - **UML packages** are used to represent an aggregation of modules. UML packages can represent, for example, layers, subsystems, and collections of implementation units that live together in the implementation namespace.
  - UML elements and relations are generic. Therefore, **stereotypes** can be used to **specialize** the **UML symbols**. A stereotype is a UML extension mechanism and is represented in diagrams as a label in guillemets («stereotype label»).

6.2. Module Views

Notations for Module Views

- **Unified Modeling Language (UML)**

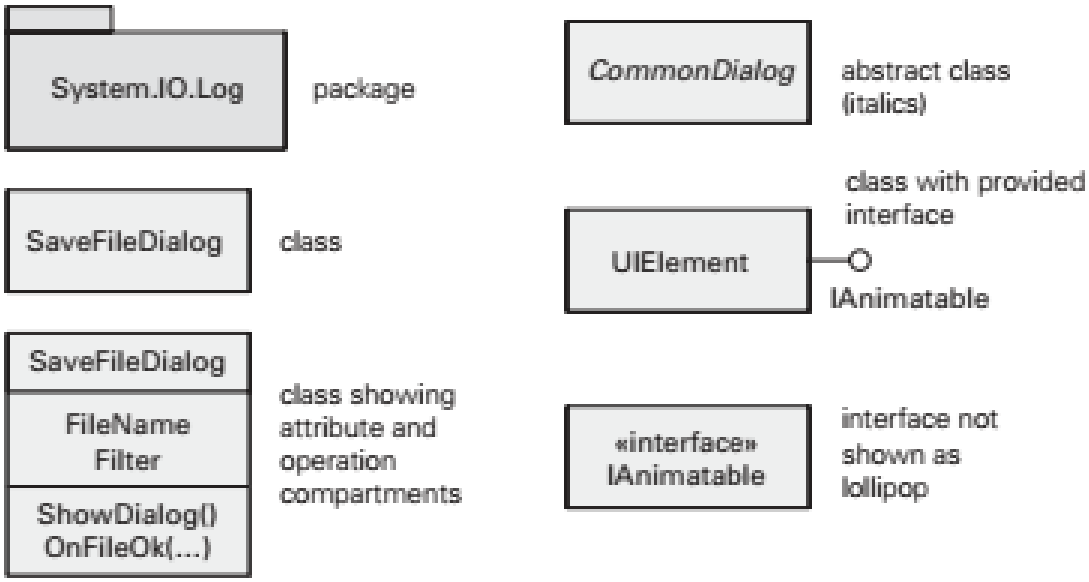


Figure 1.2
Examples of module notation in UML. A module may be represented as a class or a package. More specific types of modules can be indicated with stereotypes (as in Figure 1.4).

6.2. Module Views

Notations for Module Views

- **Unified Modeling Language (UML)**

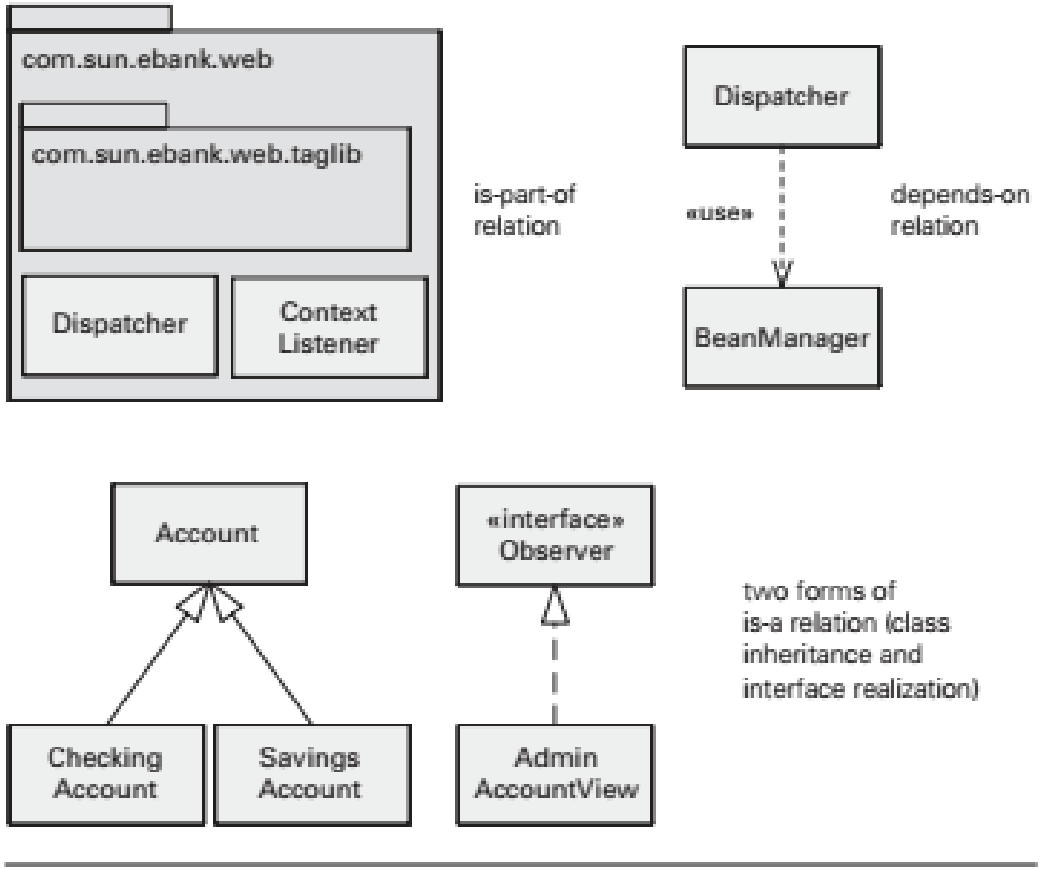


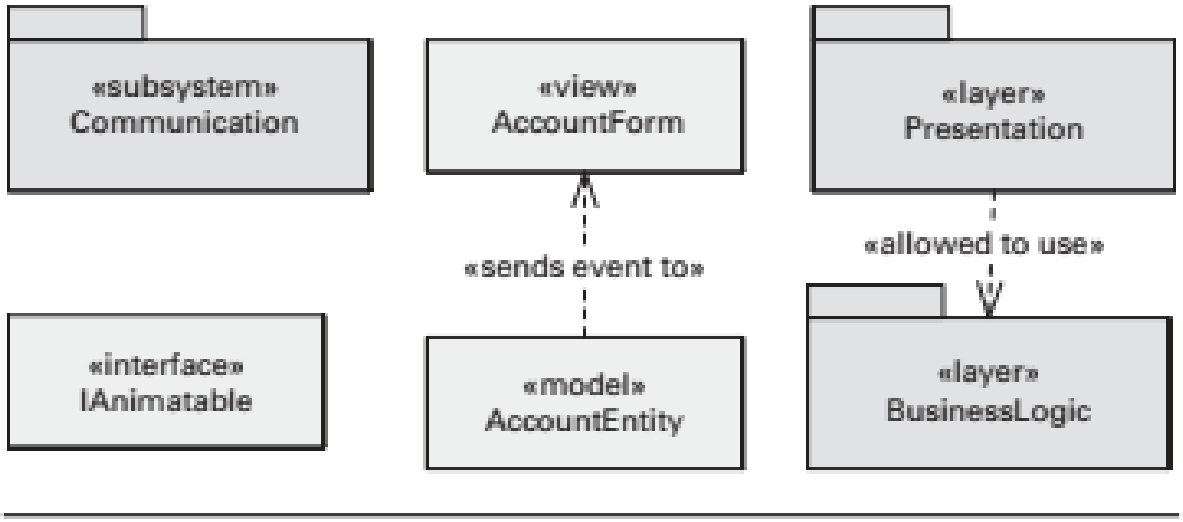
Figure 1.3
Examples of module relations in UML

6.2. Module Views

Notations for Module Views

- **Unified Modeling Language (UML)**

Figure 1.4
Examples of UML elements and relations with stereotypes



6.2. Module Views



Notations for Module Views

- **Dependency Structure Matrix**
- A dependency structure matrix (**DSM**) is a **table** that **shows modules as the column** and **row headings and dependencies as the table cells**. The DSM is **built as a square matrix** (that is, a matrix with same number of rows and columns) where element ij is nonzero if there is a dependency between module i and module j in the architecture.
- **Entity-Relationship Diagram**
- An entity-relationship diagram (**ERD**) is a notation **specifically used for data modeling**. It shows **data entities** that require a representation in the system and **their relationships**.

6.2. Module Views

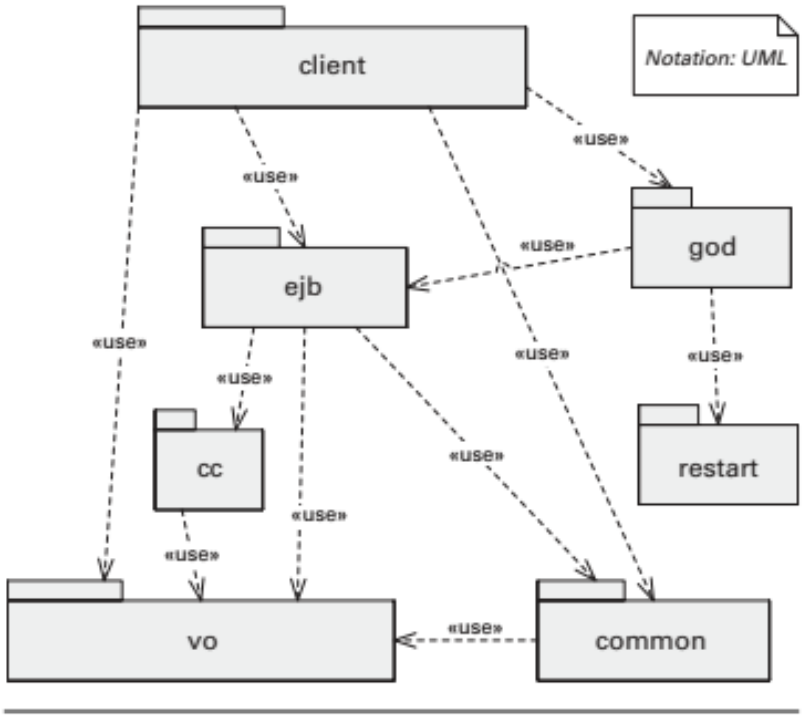
Notations for Module Views

Figure 2.9
DSM for the UML diagram in Figure 2.8

used module \ using module	client	ejb	cc	god	restart	common	vo
client	0	0	0	0	0	0	0
ejb	**1**	0	0	**1**	0	0	0
cc	0	**1**	0	0	0	0	0
god	**1**	0	0	0	0	0	0
restart	0	0	0	**1**	0	0	0
common	**1**	**1**	0	0	0	0	0
vo	**1**	**1**	**1**	0	0	**1**	0

Key: "1" means module in column uses module in row

Figure 2.8
UML package diagram showing <<uses>> dependencies

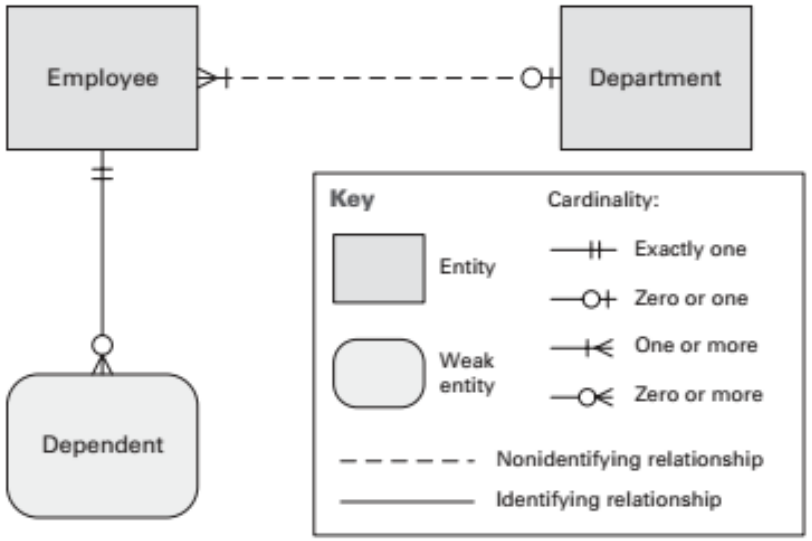


6.2. Module Views

Notations for Module Views



Figure 2.43
Data model (simplified) of a human resource system using crow's foot ERD notation



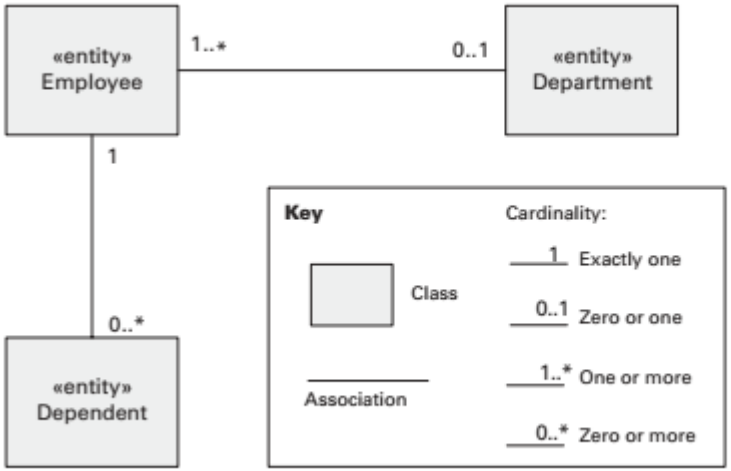


Figure 2.44
Data model (simplified) of a human resource system shown as a UML class diagram

6.2. Module Views

Notations for Module Views

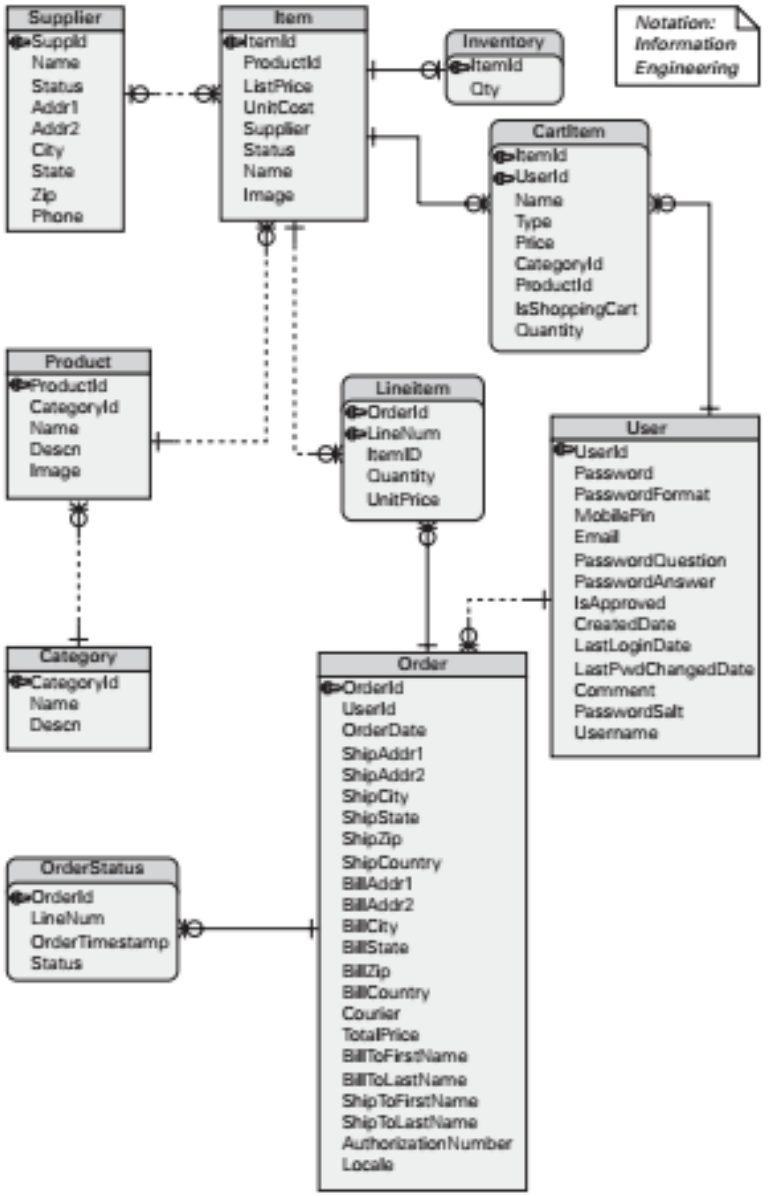


Figure 2.45
Data model for the Pet Shop application using Information Engineering crow's foot ERD notation

6.3. Component-and-Connector Views



- Component-and-Connector (C&C) views document the **system's units of execution**.
- A C&C view shows elements that have some runtime presence, such as **processes**, **objects**, **clients**, **servers**, and **data stores**. These elements are called **components**.
- Additionally, C&C views include as elements the **pathways of interaction**, such as **communication links** and **protocols**, **information flows**, and **access to shared storage**. Such interactions are represented as **connectors** in C&C views.

6.3. Component-and-Connector Views

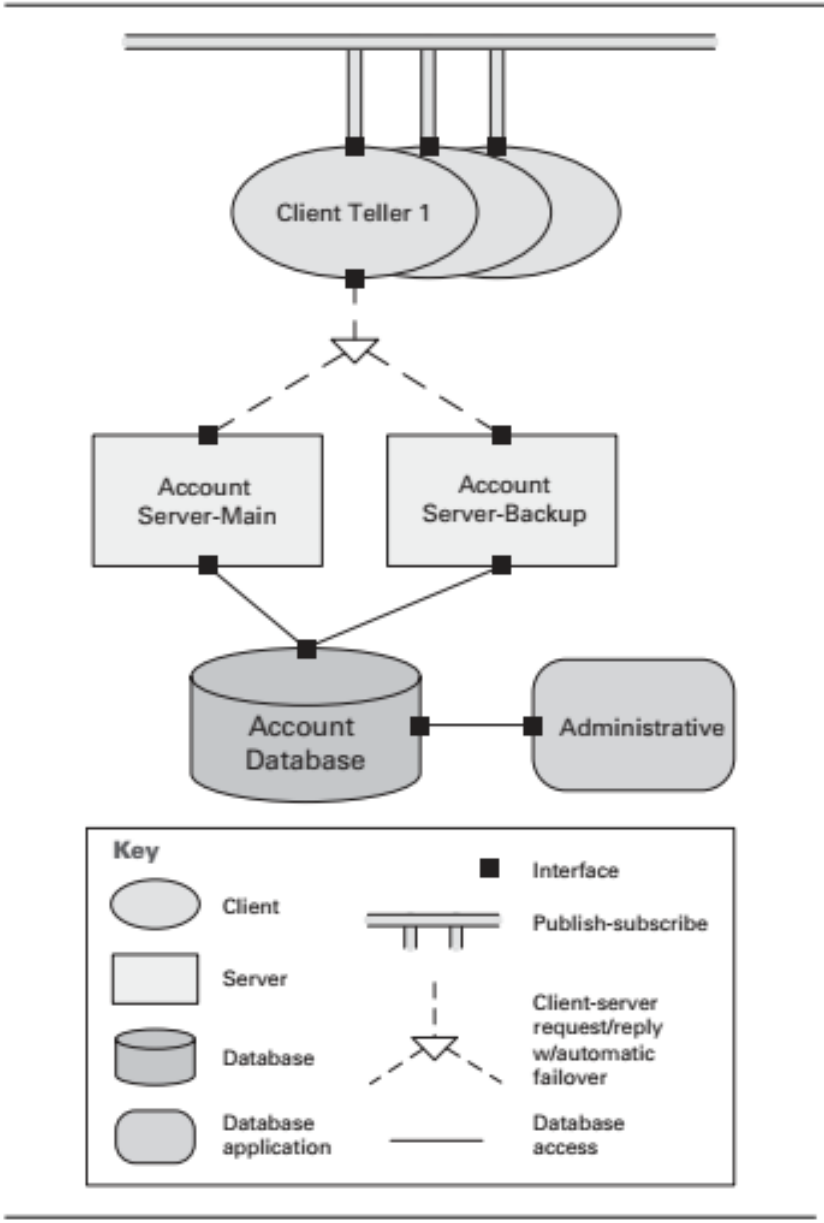


Figure 3.1
A bird's-eye-view of a system as it appears at runtime. This system contains a shared repository that is accessed by servers and an administrative component. A set of client tellers can interact with the account servers and communicate among themselves through a publish-subscribe connector.

6.3. Component-and-Connector Views



- Each of the three types of connectors shown in Figure 3.1 represents a **different form of interaction** among the connected parts.
  - **Client-server connectors** allow a set of concurrent clients to retrieve data synchronously via service requests. This variant of the client-server style supports transparent failover to a backup server.
  - The **database access connector** supports transactional, authenticated access for reading, writing, and monitoring the database.
  - The **publish-subscribe connector** supports asynchronous event announcement and notification.

6.3. Component-and-Connector Views



Table 3.1 Summary of C&C views

Elements	• *Components*: principal processing units and data stores. A component has a set of *ports* through which it interacts with other components (via connectors). • *Connectors*: pathways of interaction between components. Connectors have a set of roles that indicate how components may use a connector in interactions.
Relations	• *Attachments*: component ports are associated with connector roles to yield a graph of components and connectors. • *Interface delegation*: in some situations component ports are associated with one or more ports in an "internal" subarchitecture. Similarly for the roles of a connector.
Constraints	• Components can be attached only to connectors, not other components. • Connectors can be attached only to components, not other connectors. • Attachments can be made only between compatible ports and roles. • Interface delegation can be defined only between two compatible ports (or two compatible roles). • Connectors cannot appear in isolation; a connector must be attached to a component.
What C&C Views Are For	• Showing how the system works • Guiding development by specifying the structure and behavior of runtime elements • Helping architects and others to reason about runtime system quality attributes, such as performance, reliability, and availability

6.3. Component-and-Connector Views

Component-and-Connector Views

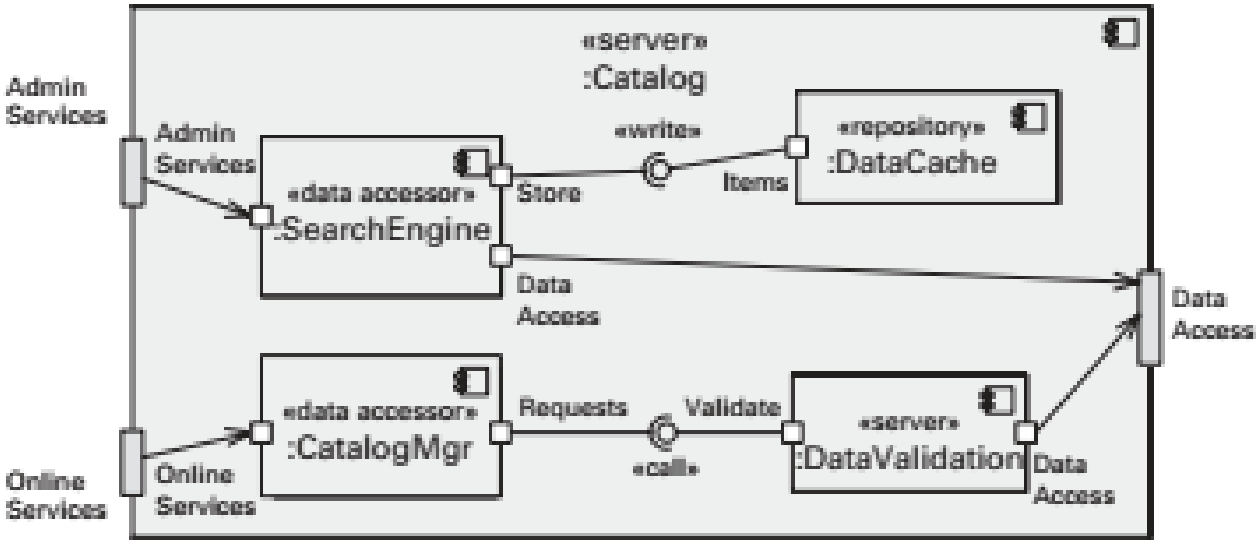


Figure 3.2
A UML component diagram showing the subarchitecture of a component called `Catalog`. UML delegation connectors associate the ports of `Catalog` with the ports of internal components.

6.3. Component-and-Connector Views

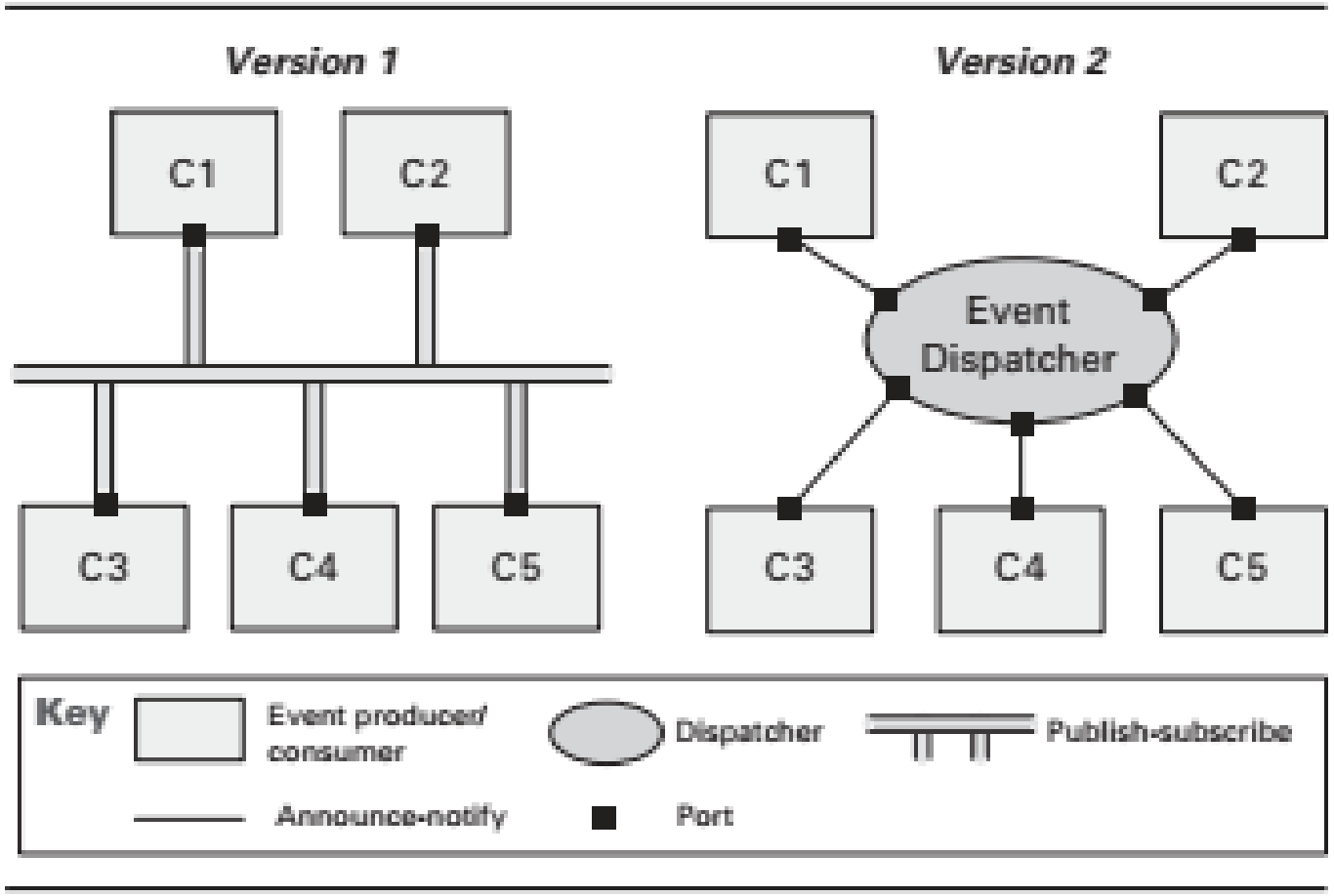


Figure 3.5
Two potential versions of a publish-subscribe system. In Version 1, all communication takes place over an event bus; in Version 2, communication occurs with the assistance of a dispatcher component.

6.3. Component-and-Connector Views



- **Connectors** can represent **complex abstractions**; they have types and interfaces, or roles; and they **require detailed semantic documentation**.

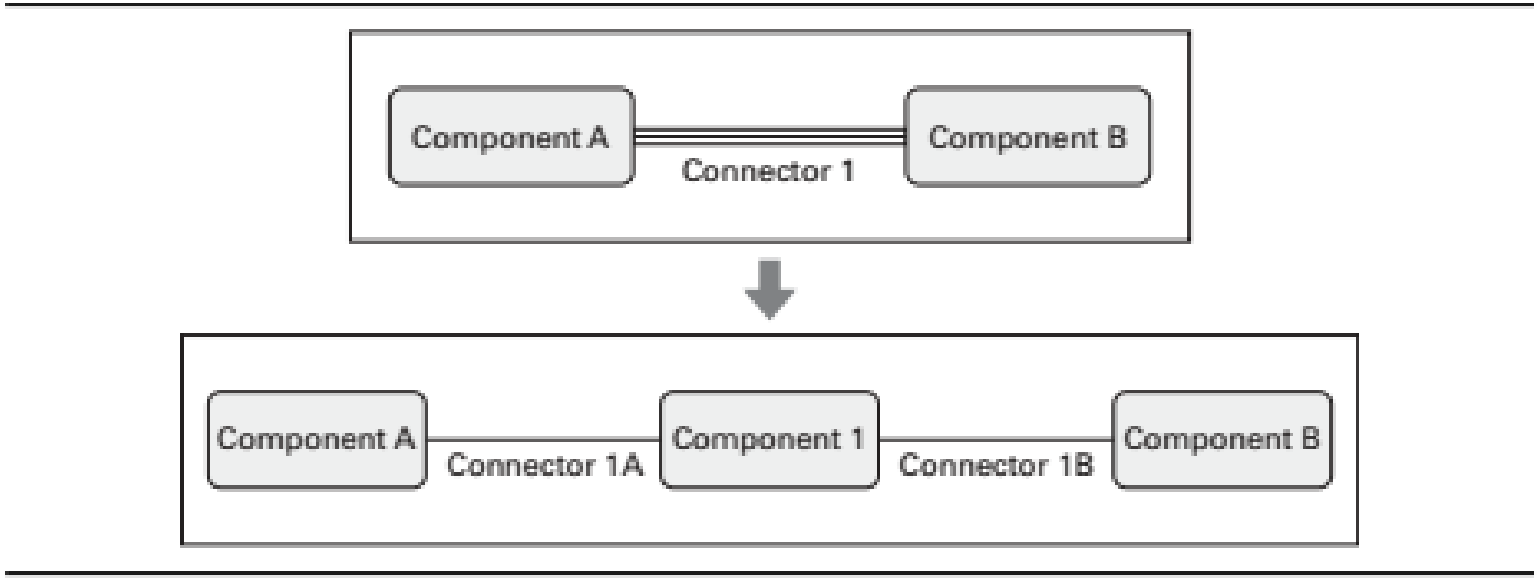


Figure 3.4
A complex connector and the alternative of representing it as a component with two simpler connectors

6.3. Component-and-Connector Views



Notations for C&C Views

- **Informal Notations**
- **Box-and-line drawings** are available to represent C&C views.
- The **primary guideline** is to **assign each component type** and **each connector type** a **separate visual form** (**symbol**), and to **list each of the types in a key**.
- **Beyond just naming the types**, however, **their meaning should be specified**.
- **Take special care with connectors**. A **common source of ambiguity** in most existing architecture documents is the **meaning of connectors**, especially ones that use arrows as their visual symbol. Make sure to say what the arrow's direction means.

6.3. Component-and-Connector Views



Notations for C&C Views

- **Formal Notations**
- Most, if not all, **architecture description languages** (**ADLs**) can be used to describe component-and-connector types, **constraints on topologies of component-and-connector graphs**, and **properties that can be associated with the elements** of the graph. **Tools** may then process an architecture description by referring to the **meanings** of the **types**, the **constraints**, and the **properties**.

6.3. Component-and-Connector Views



Notations for C&C Views

- **Semiformal Notations: UML**
- **UML components** are a **good semantic match to C&C components** because they **permit intuitive documentation of important information** such as **interfaces**, **properties**, and **behavioral descriptions**. UML components also **distinguish between component types** and **component instances**, which is useful when defining view-specific component types.
- Because C&C components that appear in a view are instances, they should be represented using **UML component instances**.

6.3. Component-and-Connector Views

Notations for C&C Views

- **Semiformal Notations: UML components**

Figure 3.6
A UML representation of a portion of the C&C view originally presented in Figure 3.1. This fragment only shows how four components are represented in UML. `Main` and `Backup` are instances of the same component type (`Account Server`).

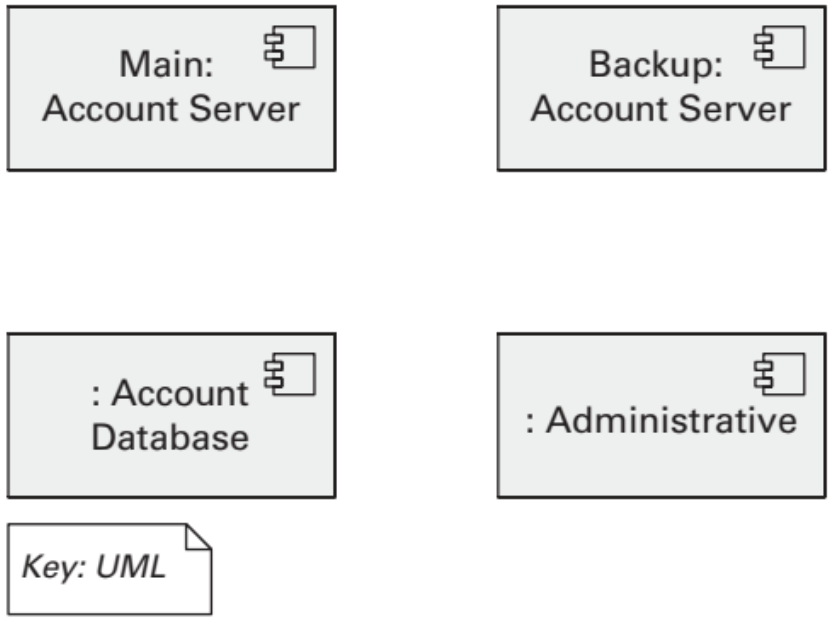
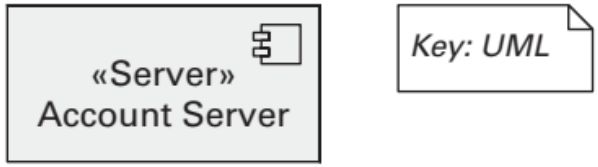


Figure 3.7
A UML representation of a C&C component type. The `Account Server` component type is a specialization of the Server component type from the client-server style (see Section 4.3.1).



6.3. Component-and-Connector Views

Notations for C&C Views

- **Semiformal Notations: UML components**

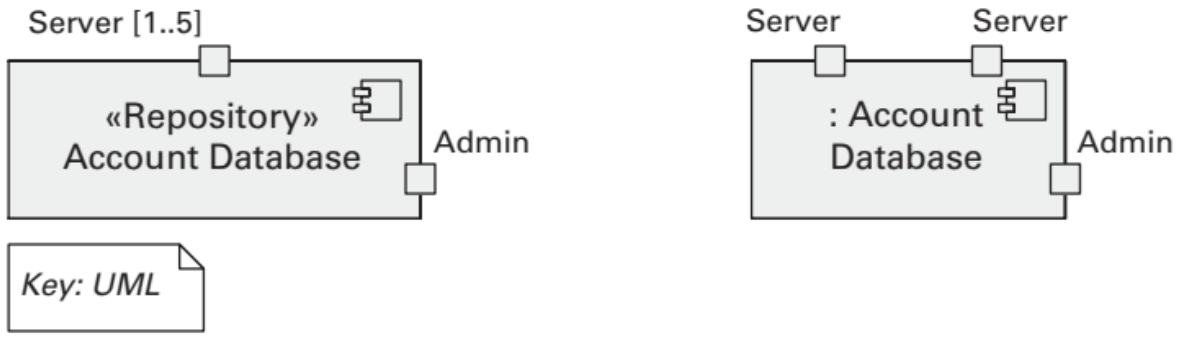


Figure 3.8
A UML representation of the ports on a C&C component type (left) and a component instance (right). The `Account Database` component type has two types of ports, `Server` and `Admin` (denoted by the boxes on the component's border). The `Server` port is defined with a multiplicity, meaning that multiple instances of the port are permitted on any corresponding component instance.

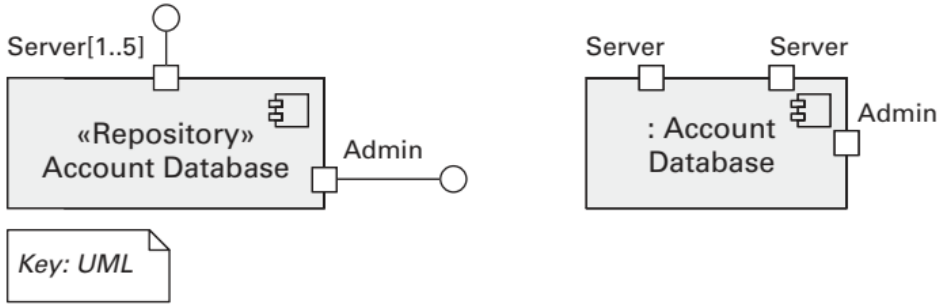


Figure 3.9
Each port on the `Account Database` type now includes one supplied interface (the lollipop), which can be further elaborated in UML by supplying additional information, such as methods or attributes. The instance of `Account Database` on the right has exactly two `Server` ports, and the interfaces are omitted.

6.3. Component-and-Connector Views



Notations for C&C Views

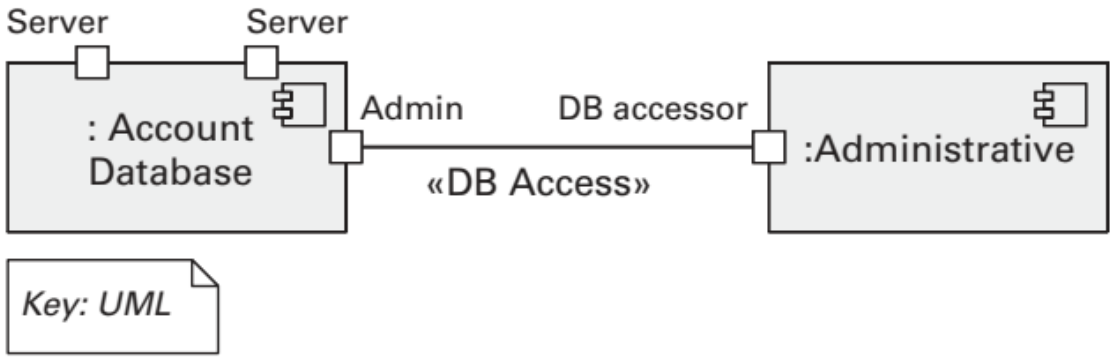
- **Semiformal Notations: UML**
- **Connectors in UML**
- While **C&C connectors** are as **semantically rich** as C&C components, **UML connectors are not always rich enough**. UML connectors cannot have substructure, attributes, or behavioral descriptions.
- You **should represent** a **"simple" C&C connector using** a **UML connector—a straight line**.
- You can use a **stereotype to denote** the **type of connector**.
- **Connector roles cannot be explicitly represented with a UML connector**. The best approximation is to **label** the **connector ends** and **use these labels to identify role descriptions** that **must be documented elsewhere**.

6.3. Component-and-Connector Views

Notations for C&C Views

- **Semiformal Notations: UML**
- **Connectors in UML**
- If you also need to supply **simple descriptive information**, such as **attribute-value pairs**, attach it to a **UML connector by using tagged values or a comment**.

Figure 3.10
A UML representation of a "simple" C&C connector between two components. The type of the connector is noted by a stereotype (<<DB Access>> in this case).



6.3. Component-and-Connector Views

Notations for C&C Views

- **Semiformal Notations: UML**
- **Connectors in UML**
- You should represent a **“rich” C&C connector using a UML component**, or by **annotating** a **straight-line UML connector with a tag or other auxiliary documentation** that **explains** the **meaning** of the complex connector.

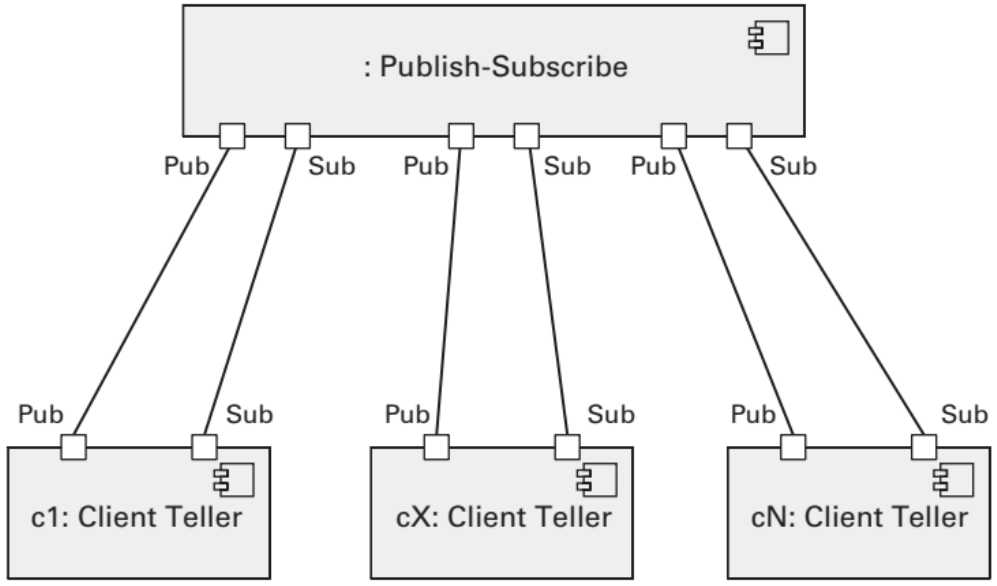


Figure 3.11
A UML representation of a “rich” C&C connector used to connect three components. The Publish-Subscribe connector is represented using a UML component. Its roles are represented using UML ports. Attachments between C&C ports and roles is represented using UML connectors between the respective UML ports.

6.3. Component-a

Notations for C&C Views

- **Semiformal Notations: UML**

ADVICE

UML for C&C

- Use UML components and ports to model C&C components and ports.
- Always show a component's ports explicitly, even though UML doesn't require it.
- Use a <<stereotype>> to indicate the type of a component or connector instance in a view, if that type was defined in a style guide. If the type is specific to a view, its name appears after the colon in the instance name.
- Represent a simple C&C connector with a straight-line UML connector or (if it's a call-return connector) with a UML assembly connector (a lollipop/socket pair).
- Represent a more complex C&C connector as a UML component, possibly with substructure, or with a straight-line UML connector annotated by a tag that explains the meaning of the connector.
- Use the lollipop/socket connector in UML only for call-return connectors. Avoid it otherwise.
- Don't attach connectors directly to a component; attach connectors to a specific port of a component.

6.4. Allocation Views



- **Software elements** in a software architecture **interact with nonsoftware elements in** the **environment** in which the software is developed, deployed, and executed. **Computing and communication hardware**, **file management systems**, and **development teams** all interact with the software architecture.
- **Allocation views** present a **mapping between software elements** and **nonsoftware elements in** the **software's environment**.

6.4. Allocation Views



- ***Three common allocation styles***:
  - The ***deployment style*** describes the **mapping** between the **software's components** and **connectors** and the **hardware** of the computing platform on which the software executes.
  - The ***install style*** describes the **mapping** between the **software's components** and **structures** in the **file system** of the production environment.
  - The ***work assignment style*** describes the **mapping** between the **software's modules** and the **people**, **teams**, or **organizational work units** tasked with the development of those modules.

6.4. Allocation Views



Allocation styles

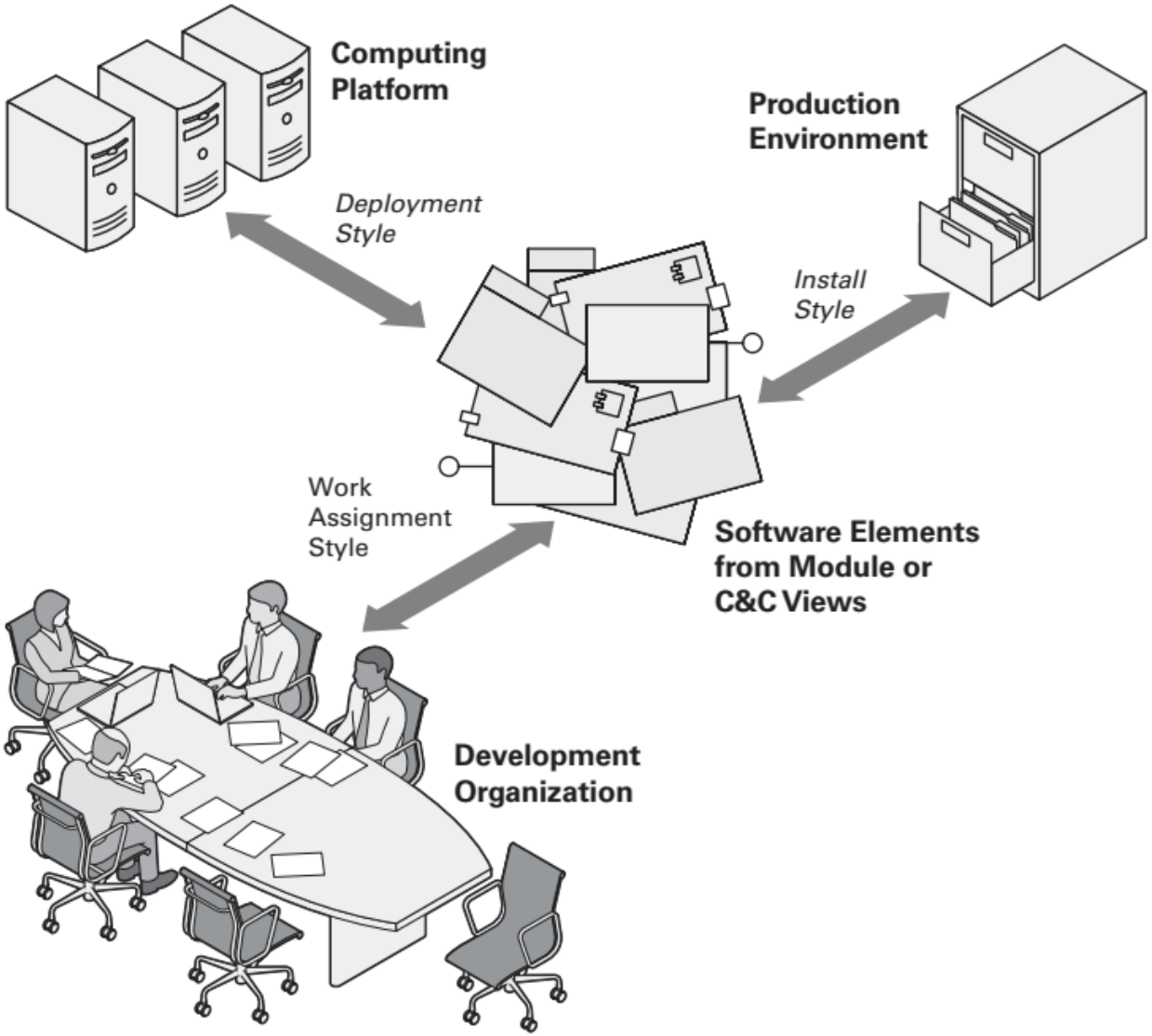
Table 5.1 Summary of the characteristics of the allocation styles

Overview	Allocation styles describe the mapping between the software architecture and its environment.
Elements	*Software element* and *environmental element*. A software element has properties that are *required* of the environment. An environmental element has properties that are *provided* to the software.
Relations	*Allocated-to*. A software element is mapped (allocated to) an environmental element. Properties are dependent on the particular style.
Constraints	Varies by style

6.4. Allocation Views



Figure 5.1
Three allocation styles are deployment (mapping software architecture to the hardware of the computing platform), install (mapping it to a file system in the production environment), and work assignment (mapping it to the teams in the development organization).



6.4. Allocation Views



- **Deployment style**
- In the **deployment style**, **software elements** native to a **C&C style are allocated to** the **hardware of** the **computing platform on which** the **software executes**. A valid allocation ensures that the **requirements expressed by** the **software elements are satisfied by** the **characteristics of** the **hardware element**(s).

6.4. Allocation Views



Deployment style

Table 5.2 Summary of the deployment style

Overview	The deployment style describes the mapping of components and connectors in the software architecture to the hardware of the computing platform.
Elements	• *Software element: elements from a C&C view.* Useful properties to document include the significant features required from hardware, such as processing, memory, capacity requirements, and fault tolerance. • *Environmental elements: hardware of the computing platform—processor, memory, disk, network (such as router, bandwidth, firewall, bridge), and so on.* Useful properties of an environmental element are the significant hardware aspects that influence the allocation decision.
Relations	• *Allocated-to*. Physical units on which the software elements reside during execution. Properties include whether the allocation can change at execution time or not. • *Migrates-to*, *copy-migrates-to*, and/or *execution-migrates-to* if the allocation is dynamic. Properties include the trigger that causes the migration.
Constraints	The allocation topology is unrestricted. However, the required properties of the software must be satisfied by the provided properties of the hardware.

6.4. Allocation Views



- The **important properties of the elements** are the **significant hardware features** that **affect the allocation** of the software to the physical units.
- **How** a **physical unit satisfies** a **software element requirement** is **determined by** the **properties of both**.
- The **types of analyses** to be **performed** via a deployment view also **determine** the **particular properties** the **elements must possess**.

6.4. Allocation Views



- Some **environmental element properties** relevant to physical units:
  - **CPU properties**. The properties relevant to the various processing elements (such as processor clock speed, number of processors, memory capacity, bus speed, cache size, and instruction execution speed).
  - **Memory properties**. The properties relevant to the memory stores (such as memory size and speed characteristics).
  - **Disk or other storage unit capacity**. The storage capacity and access speed of disk units: individual disk drives, disk farms, and redundant arrays of independent disks (RAIDs).
  - **Bandwidth**. The data transfer capacity of communication channels.
  - **Fault tolerance**. Multiple hardware units may perform the same function, and these units may have a failover control mechanism.

6.4. Allocation Views



- **Properties** that are **relevant to software elements**:
  - **Resource consumption**. For example, a computation takes 32,123 instructions always, or at most, or on average, or under nominal (error-free) conditions, and so on.
  - **Resource requirements and constraints that must be satisfied**. For example, a software element must execute in no more than 0.1 second.
  - **Safety critical**. For example, this would be true if a software element must always be running.
- The following **property** is **relevant to** the **allocation**:
  - **Migration trigger**. If the allocation can change as the system executes, this property specifies what must occur for a migration of a software element from one processing element to another.

6.4. Allocation Views



- **What the Deployment Style Is For**:
  - A deployment view is **useful for analyzing performance**, **availability**, **reliability**, and **security**. **Testers** use this view to **understand runtime dependencies**, and **integrators** use it to **plan integration** and **integration testing**. A deployment view may also be used to **support cost estimation** when evaluating purchasing options for hardware.

6.4. Allocation Views



- **Notation** for the Deployment Style:
  - **Informal Notations**
    - **Informal graphical notations contain boxes, circles, lines, arrows**, and so on to represent the software and environmental elements.
    - If the deployment structure is simple, a **table** that **lists** the **software units** and the **hardware element** on which each executes may be adequate.
  - **Formal Notations**
    - The Architecture Analysis and Design Language (**AADL**) and **SysML** are examples of architecture description languages.
  - **UML**
    - In UML, a **deployment diagram** is a graph of nodes connected by communication associations.

6.4. Alloca

Notation for the Deployment Style

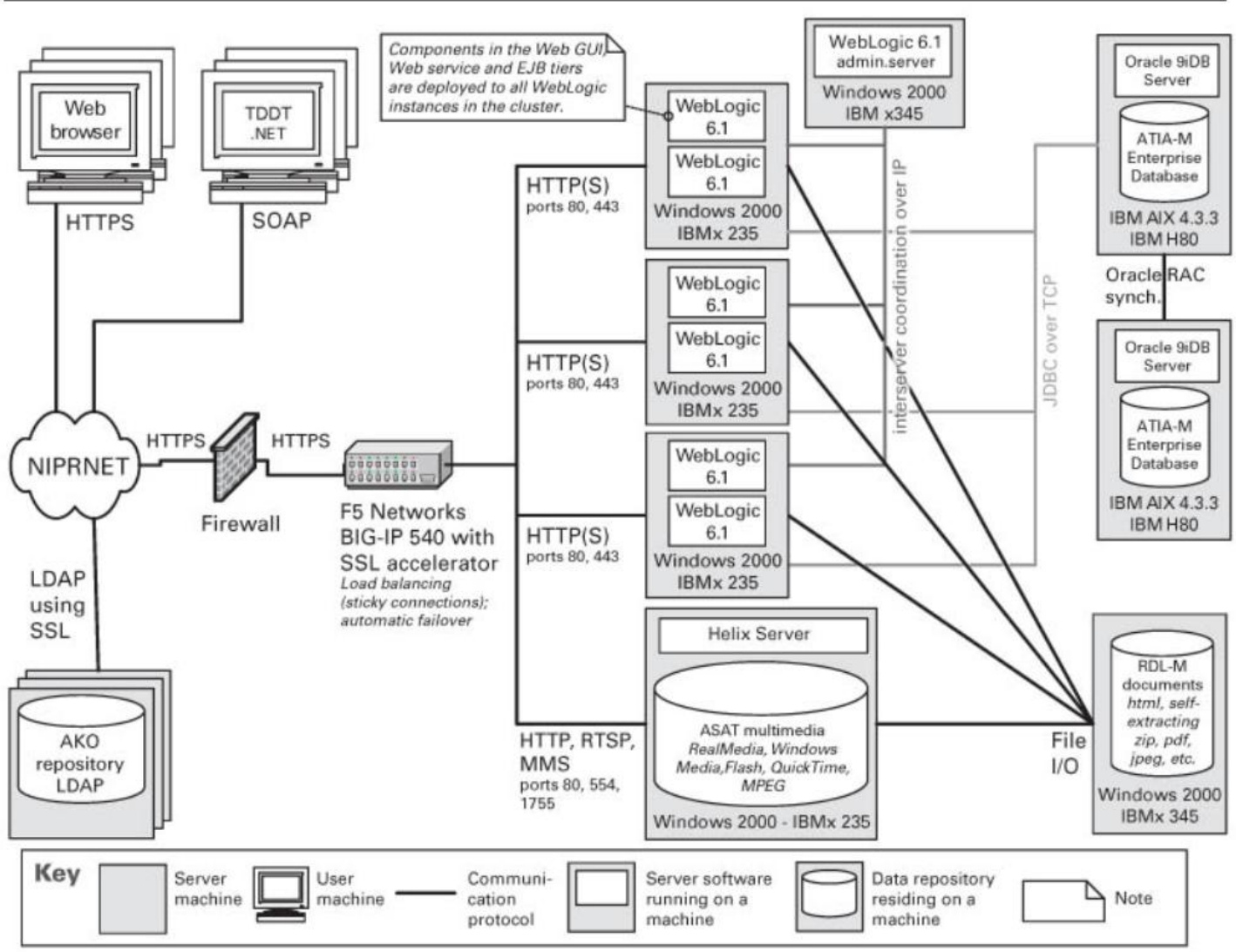


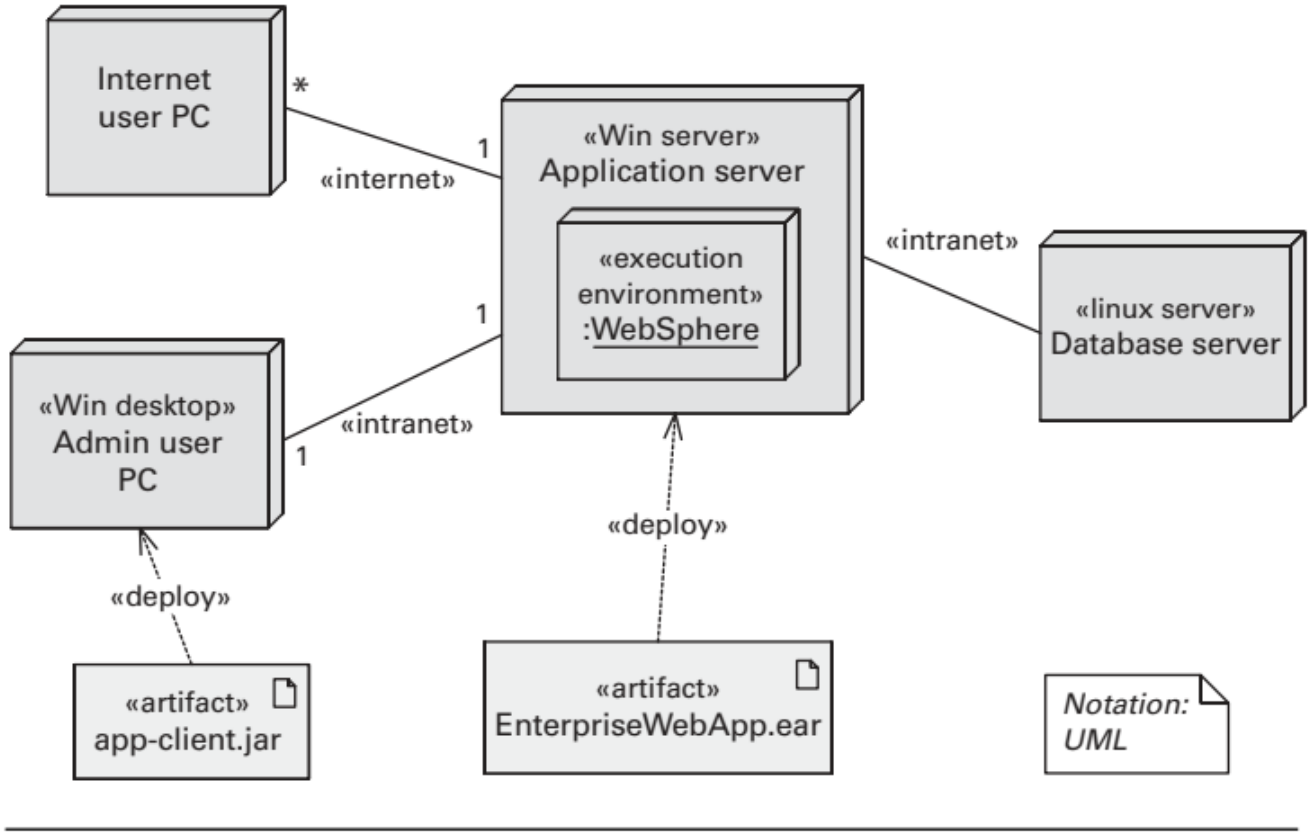
Figure 5.2
Example of a deployment view in an informal notation. This example comes from the U.S. Army Training Information Architecture-Migrated (ATIA-M) System and uses distinctive symbols for different types of hardware. The connecting lines are physical communication channels that allow the components to communicate with one another. The allocation of components is shown by overlaying their names on the symbol. The allocation of connectors is done by writing their names adjacent to the channels to denote the communication protocol. ATIA is a Java Platform, Enterprise Edition (Java EE) application comprising hundreds of components (mostly servlets and Enterprise JavaBeans [EJBs]). The ATIA architecture has a client-server multi-tier view with a Web GUI tier, a Web service tier, and an EJB tier. All components inside those tiers are deployed to WebLogic, as indicated by the annotation. NIPRNET is an Internet-like network owned by the Department of Defense.

6.4. Allocation Views

Notation for the Deployment Style



Figure 5.3
A deployment view in UML, showing the hardware platform supporting a Java EE system. The <<deploy>> dependency shows which artifacts are deployed to which nodes. <<execution environment>> is a node that offers an environment to run specific types of components. To know what components are deployed to a specific node, you need to look at the install view to see what components go inside each artifact.



6.4. Allocation Views



- **Install style**
- The **install style allocates components of a C&C style to a file management system in the production environment**. Once a software system is implemented, the resulting files have to be packaged to be installed on the target production platform (such as a desktop computer or a server machine running an application server). **These files include libraries**, **executable files**, **data files**, **log files**, **configuration** and **version control files**, **license files**, **help files**, **deployment descriptors**, **scripts**, and **static content** (for example, HTML files and images).
- The install style **helps describe what specific files should be used** and **how they should be configured** and **packaged to produce different versions of the system**. Maintaining multiple versions simultaneously is a common practice for many systems.
- The **important properties** of the software and environmental elements of the install style **are those that affect the allocation of the software to configuration items**.

6.4. Allocation Views



Install style

Table 5.3 Summary of the install style

Overview	The install style describes the mapping of components in the software architecture to a file system in the production environment.
Elements	• *Software element: a C&C component*. *Required* properties of a software element, if any, usually include requirements on the production environments, such as a requirement to support Java or a database, or specific permissions on the file system. • *Environmental element: a configuration item, such as a file or a folder. Provided* properties of an environmental element include indications of the characteristics provided by the production environments.
Relations	• *Allocated-to*. A component is allocated to a configuration item. • *Containment*. One configuration item is contained in another.
Constraints	Files and folders are organized in a tree structure, following an *is-contained-in* relation.

6.4. Allocation Views



- **What the Install Style Is For**: Understanding the organization of the files and folders of the installed software can help developers, deployers, and operators carry out the following tasks:
  - **Create build-and-deploy procedures**.
  - **Navigate** through a **large number of files** and **folders** that **constitute** the **installed system**, to locate specific files that require attention (such as a log file or configuration file).
  - **Select** and **configure files** to **package** a **specific version of** a **software product line**.
  - **Update** and **configure files of multiple installed versions** of the same system.
  - **Identify** the **purpose** or **contents** of a **missing** or **damaged file**, which is causing a problem in production.
  - **Design** and **implement** an **“automatic updates” feature**.

6.4. Allocation Views

Notations for the Install style

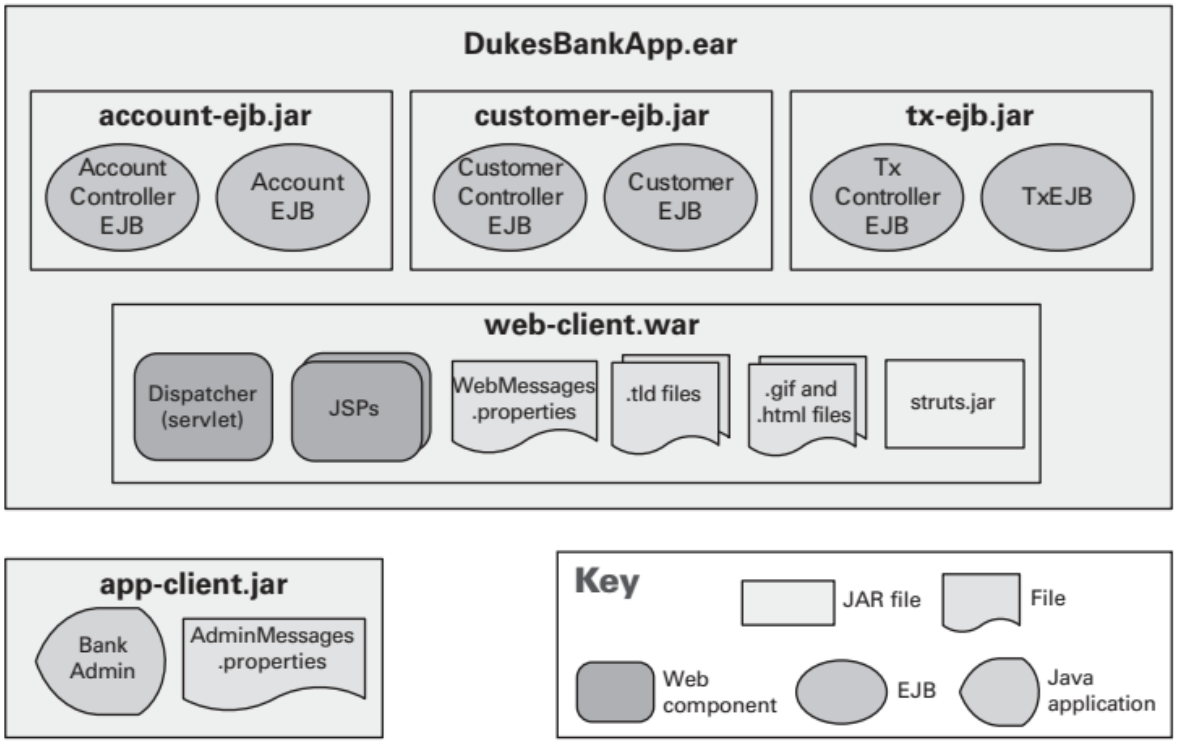


Figure 5.4
An install view in an informal notation, from the Duke's Bank Java EE application. Java applications are usually deployed in Java archive (JAR) files. Like a zip file, a JAR file may contain other files. Enterprise JavaBean JAR files contain EJB classes and other files that the EJBs may need. Web archive (WAR) files contain Web components (servlets and JavaServer Pages [JSPs]); very often, they also contain HTML, JPEG, and other files used in Web pages for "static content." Enterprise archive (EAR) files are a packaging of zero or more JAR and zero or more WAR files. All server-side components are inside DukesBankApp.ear, which is deployed to the application server. The diagram also shows that the client-side BankAdmin Java application is deployed in app-client.jar, which is deployed to the admin user's machine.

6.4. Allocation Views

Notations for the Install style

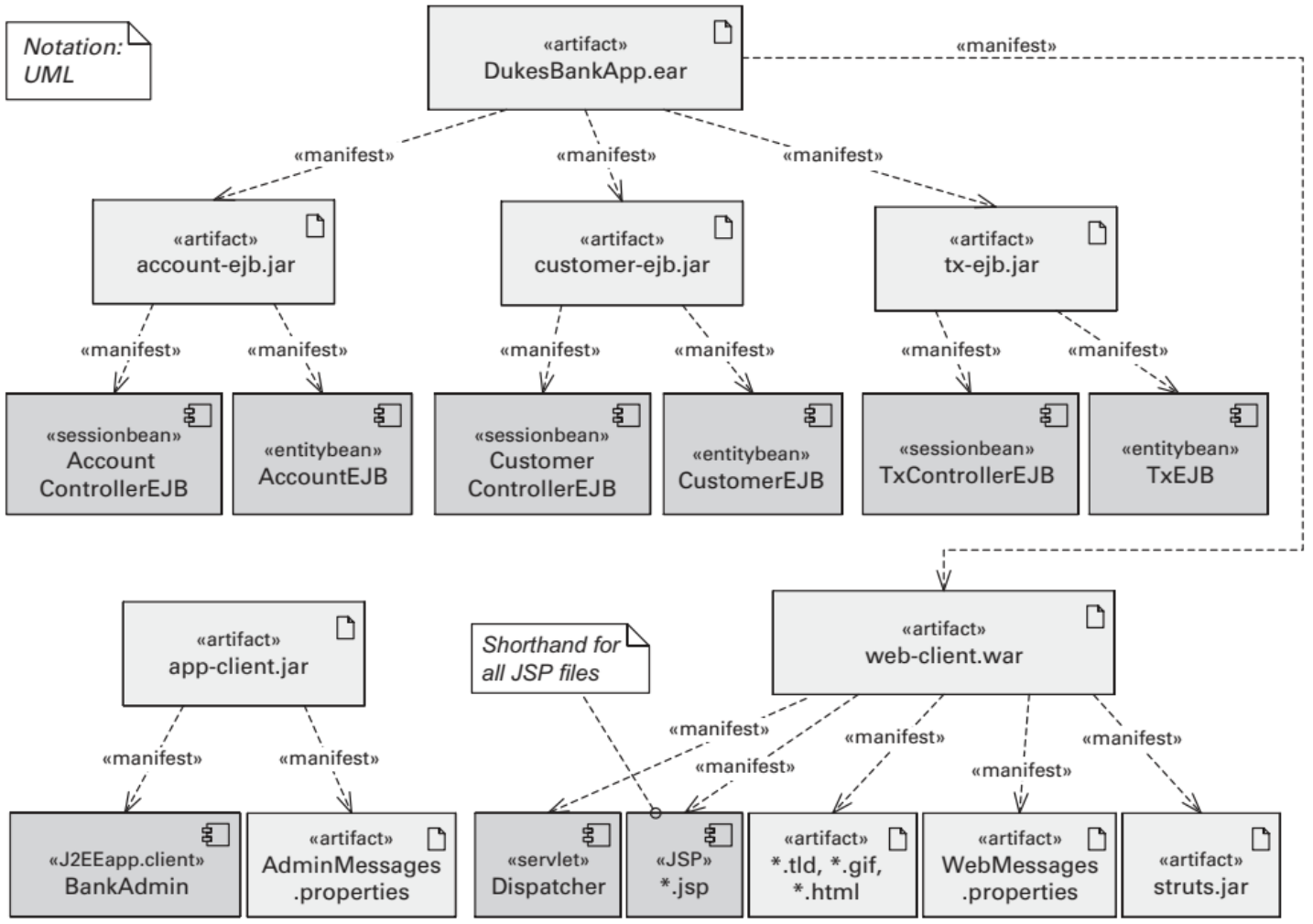


Figure 5.5
The install view of Figure 5.4 rendered in UML. The <<artifact>> stereotype denotes a file of any kind. The <<manifest>> stereotype indicates that a given component, class, or other artifact is inside a given artifact.

6.4. Allocation Views



- **Work assignment style**
- The **work assignment style allocates modules of a module style to the groups and individuals who are responsible for the realization of a system**. This style **defines** the **responsibility for implementing** and **integrating** the modules to the appropriate development teams. The style is typically used to link activities to resources to ensure that the modules are each assigned to an individual or team. The architecture in combination with process determines the actual allocations.

6.4. Allocation Views



Work Assignment Style

Table 5.4 Summary of the work assignment style

Overview	The work assignment style describes the mapping of the software architecture to the teams in the development organization.
Elements	• *Software element: a module.* Properties include the required skill set and available capacity (effort, time) needed. • *Environmental element: an organizational unit, such as a person, a team, a department, a subcontractor, and so on.* Properties include the provided skill set and the capacity in terms of labor and calendar time available.
Relations	*Allocated-to*. A software element is allocated to an organizational unit.
Constraints	In general, the allocation is unrestricted; in practice, it is usually restricted so that one module is allocated to one organizational unit.

6.4. Allocation Views



- **What a Work Assignment Style Is For**:
- The work assignment style **shows** the **major units of software** that must be present to form a working system and **who will produce them**, as well as the **tools** and **environments in which the software is developed** (and their assignments to environmental elements). The work assignment style **helps with planning** and **managing team resource allocations**, **assigning responsibilities for builds**, and **explaining the structure of a project—** to a new hire, for example. The work assignment style can give each team its charter.

6.4. Allocation Views



Notations for the Work Assignment Style

ECS Element (Module)		
Segment	**Subsystem**	**Organizational Unit**
Science Data Processing Segment (SDPS)	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment (FOS)	Planning and Scheduling	Orbital vehicle team
	Data Management	Database team
	User Interface	User interface team
. . .	. . .	. . .

Figure 5.6
Work assignment view using a tabular notation. The left two columns echo the system's module decomposition structure.



Q&A